

Assignment 1 Memo: Tech Jobs Corpus Evaluation

William Sun (yiqings@andrew.cmu.edu)

AI Disclosure: Claude (Anthropic) assisted with corpus record formatting. Phi-4-mini-instruct was used as the SLM for extraction evaluation. GPT-5-mini was used for the optional bonus comparison.

1 Human vs. SLM (Phi)

Phi achieved a 4% all-fields-correct rate with a 72% schema violation rate across the 25-record evaluation sample.

Field	Accuracy	Errors
role_category	0.92	2
company	0.76	6
salary_min	0.76	6
location_type	0.60	10
seniority	0.32	17
required_skills	0.04	24

Table 1: Phi v1 field-level accuracy on 25 evaluation records.

For `required_skills`, precision was 0.069, recall 0.333, and F1 0.114.

Phi did well on `role_category` and `company`, which are usually written out directly in the document. The two main failure modes were:

1. required_skills: rewording instead of extracting. Phi returned full descriptions like “familiarity with Python, C++, C#, or similar object-oriented programming languages” instead of short skill names like “Python”, “C++”, “C#”. It treated the field as a summary task rather than extraction. On records with no skills mentioned (company overviews, compensation tables), Phi made up generic skills like “programming languages” and “problem-solving”.

2. seniority: misclassifying new-grad roles. Phi labeled entry-level roles as “mid” (3 times), “intern” (1), or the invalid value “junior” (1, causing a schema violation). Phrases like “New Graduate” or “New College Grad” in role titles were not enough for Phi to pick “entry”. When the document described tasks like “design, develop, and maintain”, Phi seemed to guess mid-level based on the work complexity rather than the stated job level.

The most common schema violation was `$.salary_min: expected number, got NoneType` (5 times). When salary was missing from the document, Phi returned `null` instead of leaving the field out. One case returned a string for `salary_min`, and another invented the enum value “junior” for `seniority`.

Phi handled table-format records slightly better (7.1% accuracy) than text records (0%), likely because tables contain more structured, clearly labeled information.

2 Recovery Strategy

Two recovery prompts were tested on the 24 records that failed under the v1 baseline.

v2 (strict rules): Added enum constraints (“MUST be exactly one of...”) and explicit instructions to copy skills exactly as written, return empty lists when none are present, and leave out salary_min rather than guessing.

v3 (few-shot examples): A custom prompt targeting the two main failure modes. It included seniority mapping rules (“Roles titled ‘New Grad’, ‘Junior’, ‘Associate’, or ‘I’ are always ‘entry’”) and two input/output examples showing correct skill extraction (short technology names, not reworded sentences) and salary handling (leave out when absent, number when present).

Field	v1 (baseline)	v2 (strict)	v3 (fewshot)
seniority	29.2%	37.5%	87.5%
required_skills	0.0%	45.8%	58.3%
salary_min	75.0%	87.5%	83.3%
all_fields_correct	0	2	6
schema_violations	18/24	18/24	11/24

Table 2: Recovery experiment results on 24 previously-failed records.

The v3 few-shot prompt worked best. Seniority accuracy jumped from 29% to 88%, confirming that Phi needed specific mapping rules rather than general instructions. The two examples also cut schema violations from 75% to 46% by showing the expected JSON format. required_skills improved from 0% to 58%, though Phi still sometimes mixed in soft skills or degree requirements alongside technology names.

The v2 strict prompt improved required_skills (0% to 46%) but barely helped seniority (29% to 38%), showing that broad instructions are less effective than targeted rules for repeated classification errors. v2 also did nothing for schema violations (both 18/24), while v3 cut them nearly in half.

3 Cluster Reading

Clustering ($k = 6$) produced a low quality score of 0.038, meaning the clusters overlap heavily. This makes sense given that every document in the corpus is a new-grad tech job posting.

Cluster	n	Label	Top terms
0	46	Career-support & analytics overviews	early, support, data, career
1	15	AI/ML research roles	scientist, research, ai, nvidia
2	72	SWE responsibilities & descriptions	data, software, design, work
3	40	Junior/associate quals & compensation	qualifications, data, junior
4	9	Entry-level structured listings	entry, level, qualifications
5	68	New-grad SWE quals & compensation	engineer, software, grad, new

Table 3: Cluster summary with labels.

Only Cluster 5 had any correct extractions (9.1%, 1 out of 11 records). These are the most template-like records (new-grad SWE qualifications and compensation tables), where field values are spelled out directly. Clusters 0, 1, 2, and 3 all had 0% accuracy. No evaluation records fell into Cluster 4.

Cluster 2 (SWE responsibilities, $n = 72$) was the largest and hardest for Phi: narrative job descriptions rarely contain explicit skill names or salary figures, forcing Phi to guess or make things up. Cluster 1 (AI/ML research, $n = 15$) was a different problem: specialized phrasing like “generative AI for physical AI” sometimes got misclassified under `role_category`. The pattern is that Phi’s accuracy tracks document structure: template-like, tabular records (Cluster 5) are easier to extract from than free-text descriptions (Cluster 2) or specialized research postings (Cluster 1).

4 Corpus Limitations

This corpus has several gaps that limit its usefulness for broader job market analysis. It covers only new-grad and entry-level tech positions, so it says nothing about mid-career, senior, or executive roles, or how salaries change with experience. All data was pulled on a single date (2026-09-18), so there is no way to track trends or seasonal hiring patterns. The corpus is skewed toward US-based roles from English-language job boards (Greenhouse, Ashby, Workday); international markets and platforms like LinkedIn or Indeed are missing. Many records are short fragments (median 79 characters) created by splitting one posting into multiple records, which limits the context available for extraction. Finally, the corpus only contains posting text, not application outcomes, interview processes, or actual hiring decisions.

5 Optional Bonus: SLM vs. LLM

GPT-5-mini (Azure OpenAI) was evaluated on the same 25 records using the same v1 prompt and schema.

Metric	Phi-4-mini (v1)	GPT-5-mini
<code>all_fields_correct</code>	4%	16%
<code>schema_violations</code>	72%	4%
<code>role_category</code>	92%	88%
<code>seniority</code>	32%	68%
<code>required_skills</code>	4%	48%
<code>location_type</code>	60%	56%
<code>company</code>	76%	76%
<code>salary_min</code>	76%	72%

Table 4: Phi-4-mini vs. GPT-5-mini on 25 evaluation records (both using v1 prompt).

GPT-5-mini’s biggest advantage was output compliance: only 4% schema violations compared to Phi’s 72%. It consistently returned valid JSON with correct types and enum values. It also handled seniority much better (68% vs 32%), correctly reading “New Grad” and “Engineer I” titles as entry-level in most cases.

For `required_skills`, GPT-5-mini had higher precision (0.234 vs 0.069) and recall (0.733 vs 0.333) than Phi. It correctly extracted short skill names like “Python” and “C++” but still included some broad terms like “object-oriented programming” and “code reviews” that the human labels excluded.

One unexpected finding: Phi v3 (few-shot) beat GPT-5-mini on seniority (87.5% vs 68%) and `required_skills` (58.3% vs 48%) despite being a much smaller model. This suggests that targeted prompt work with domain-

specific examples can close or even reverse the gap between small and large models for structured extraction tasks. This comparison is not perfectly fair, though: v3 was evaluated only on the 24 records that failed under v1, while GPT-5-mini was evaluated on all 25. The practical takeaway is that for cost-sensitive deployments, a well-prompted SLM can compete with a larger model, though the larger model gives better schema compliance out of the box.